

LLM 数学公式参考：从 Transformer 到现代大模型推理系统

miniLLM 学习笔记

2026 年 6 月 26 日

目录

1	怎么读这份笔记	5
2	符号约定	5
3	语言建模的概率基础	5
3.1	自回归分解	5
3.2	最大似然、交叉熵和困惑度	6
3.3	Masked LM 目标	6
4	Tokenizer 与离散 token	7
4.1	BPE 合并目标	7
4.2	Unigram tokenizer	7
5	Embedding、输出投影与 tied weights	7
5.1	Token embedding	7
5.2	输出 logits	8
5.3	Weight tying	8
6	Transformer Block 总览	8
6.1	Pre-LN Transformer block	8
7	LayerNorm 与 RMSNorm	8
7.1	LayerNorm	8
7.2	RMSNorm	9

目录	2
8 Attention: 从 QK 到 Softmax	9
8.1 Q、K、V 的线性投影	9
8.2 为什么是 $QK^\top$	10
8.3 Causal mask	10
8.4 Softmax 的数值稳定形式	10
8.5 为什么 softmax/attention 内存受限	11
8.6 FlashAttention 的 online softmax 递推	11
9 Multi-Head、MQA 与 GQA	12
9.1 Multi-Head Attention	12
9.2 Multi-Query Attention	12
9.3 Grouped-Query Attention	12
10 位置编码	13
10.1 Sinusoidal positional encoding	13
10.2 RoPE	13
10.3 ALiBi	13
11 FFN、GELU 与 SwiGLU	14
11.1 标准 FFN	14
11.2 GELU	14
11.3 SwiGLU	14
12 Softmax 交叉熵的梯度	14
12.1 Label smoothing	15
13 优化器和训练动态	15
13.1 SGD	15
13.2 Adam	16
13.3 AdamW	16
13.4 Gradient clipping	16
13.5 Warmup + cosine decay	16
14 Scaling Laws 与 Chinchilla	17
14.1 经验损失分解	17
14.2 训练计算量近似	17
15 SFT、Reward Model、RLHF 和 DPO	17
15.1 Supervised Fine-Tuning	17
15.2 Reward model	18

目录	3
15.3 PPO for RLHF	18
15.4 DPO	18
16 LoRA 与参数高效微调	19
16.1 低秩适配	19
17 MoE: Dense 与 Sparse	19
17.1 Dense FFN	19
17.2 Sparse MoE	19
17.3 Switch Transformer top-1	20
17.4 负载均衡损失	20
18 推理: 从 logits 到 token	20
18.1 Temperature	20
18.2 Greedy decoding	21
18.3 Top-k sampling	21
18.4 Top-p / nucleus sampling	21
18.5 Beam search	21
19 KV Cache 与解码复杂度	21
19.1 没有 KV cache 的代价	21
19.2 有 KV cache 的增量解码	22
19.3 KV cache 显存公式	22
19.4 PagedAttention	22
20 Speculative Decoding	23
21 量化公式	23
21.1 Affine quantization	23
21.2 Symmetric quantization	23
22 分布式训练/推理的核心公式	24
22.1 Data Parallel	24
22.2 Tensor Parallel: 列切分	24
22.3 Tensor Parallel: 行切分	24
22.4 Expert Parallel	25
23 Multimodal 与 Diffusion 相关公式	25
23.1 Vision patch embedding	25
23.2 CLIP 对比学习损失	25
23.3 DDPM forward process	26

目录	4
23.4 噪声预测训练目标	26
23.5 Classifier-Free Guidance	26
24 总关联图	27
24.1 训练侧	27
24.2 推理侧	27
24.3 系统侧	27
25 公式学习建议	27

1 怎么读这份笔记

这份笔记按 LLM 的真实数据流组织公式：

文本 $\rightarrow$ Tokenizer $\rightarrow$ Token IDs $\rightarrow$ Embedding $\rightarrow$ Transformer Blocks $\rightarrow$ Logits $\rightarrow$ $\begin{cases} \text{训练: Loss + Opt} \\ \text{推理: Sampling +} \end{cases}$

其中最核心的数学链条是：

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t | x_{<t}) \implies \mathcal{L}_{\text{LM}} = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}) \implies \nabla_{\theta} \mathcal{L} \implies \theta \leftarrow \theta - \eta \cdot \widehat{\nabla_{\theta} \mathcal{L}}.$$

Transformer 只是实现条件概率 $p_{\theta}(x_t | x_{<t})$ 的函数族；vLLM/SGLang 这类推理系统则负责高效执行这个函数族。

2 符号约定

符号	形状/含义	说明
V	词表大小	vocabulary size
T	序列长度	sequence length
B	batch size	批大小
d_{model}	hidden size	模型宽度
H	attention heads	query heads 数
H_{kv}	KV heads	MHA: $H_{\text{kv}} = H$; MQA: 1; GQA: G
d_h	head dim	通常 $d_{\text{model}} = H d_h$
L	Transformer 层数	number of layers
x_t	第 t 个 token id	$x_t \in \{1, \dots, V\}$
$\mathbf{h}_t^{\ell}$	第 ℓ 层第 t 个 hidden state	$\mathbf{h}_t^{\ell} \in \mathbb{R}^{d_{\text{model}}}$
θ	模型参数	all trainable parameters

3 语言建模的概率基础

3.1 自回归分解

LLM 的预训练核心是估计 token 序列的联合概率。根据链式法则：

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t | x_{1:t-1}). \quad (1)$$

GPT 类模型使用这个自回归分解训练和生成。来源：GPT/GPT-2 系列语言模型目标 [3, 4], Transformer decoder 结构来源于 [1]。

3.2 最大似然、交叉熵和困惑度

对数据集 $\mathcal{D}$ 最大化似然：

$$\theta^* = \arg \max_{\theta} \sum_{x_{1:T} \in \mathcal{D}} \log p_{\theta}(x_{1:T}). \quad (2)$$

等价于最小化 negative log-likelihood：

$$\mathcal{L}_{\text{NLL}}(\theta) = - \sum_{x_{1:T} \in \mathcal{D}} \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}). \quad (3)$$

若对 token 平均：

$$\bar{\mathcal{L}}_{\text{NLL}} = - \frac{1}{N_{\text{tok}}} \sum_{n=1}^{N_{\text{tok}}} \log p_{\theta}(x_n \mid x_{<n}). \quad (4)$$

困惑度：

$$\text{PPL} = \exp(\bar{\mathcal{L}}_{\text{NLL}}). \quad (5)$$

解释：

- NLL 越低，模型给真实下一个 token 的概率越高。
- PPL 可理解为模型平均在多少个等概率候选中困惑；越低越好。

3.3 Masked LM 目标

BERT 使用 masked language modeling，不是纯自回归：

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log p_{\theta}(x_i \mid x_{\setminus \mathcal{M}}), \quad (6)$$

其中 $\mathcal{M}$ 是被 mask 的 token 位置集合。来源：BERT [2]。

关系：

- GPT/LLM 推理常用公式是式 (1)。
- BERT 更适合理解/编码任务；GPT 更适合逐 token 生成。

4 Tokenizer 与离散 token

4.1 BPE 合并目标

BPE 每一步选择语料中出现最频繁的相邻 symbol pair 合并：

$$(a^*, b^*) = \arg \max_{(a,b)} \text{count}_{\mathcal{D}}(a, b). \quad (7)$$

来源：subword BPE for NMT [5]。

4.2 Unigram tokenizer

SentencePiece 的 unigram 模型把一句话 $\mathbf{x}$ 的分词看成潜变量 $\mathbf{z}$ ：

$$p(\mathbf{x}) = \sum_{\mathbf{z} \in \mathcal{Z}(\mathbf{x})} \prod_{u \in \mathbf{z}} p(u). \quad (8)$$

训练时最大化语料 likelihood：

$$\mathcal{L}_{\text{tok}} = \sum_{\mathbf{x} \in \mathcal{D}} \log p(\mathbf{x}). \quad (9)$$

来源：SentencePiece / subword regularization [6, 7]。

关系：

$$\text{Tokenizer} \implies x_t \in \{1, \dots, V\} \implies \text{Embedding lookup}.$$

5 Embedding、输出投影与 tied weights

5.1 Token embedding

设 token embedding 矩阵：

$$E \in \mathbb{R}^{V \times d_{\text{model}}}. \quad (10)$$

token id x_t 对应 one-hot 向量 $\mathbf{e}_{x_t} \in \mathbb{R}^V$, embedding 为：

$$\mathbf{u}_t = E^\top \mathbf{e}_{x_t} = E[x_t] \in \mathbb{R}^{d_{\text{model}}}. \quad (11)$$

5.2 输出 logits

最后一层 hidden state $\mathbf{h}_t^L$ 投影到词表:

$$\mathbf{z}_t = W_U \mathbf{h}_t^L + \mathbf{b}_U, \quad W_U \in \mathbb{R}^{V \times d_{\text{model}}}. \quad (12)$$

概率分布:

$$p_\theta(x_{t+1} = i \mid x_{\leq t}) = \text{softmax}(\mathbf{z}_t)_i = \frac{\exp(z_{t,i})}{\sum_{j=1}^V \exp(z_{t,j})}. \quad (13)$$

5.3 Weight tying

很多 LLM 使用输入 embedding 和输出投影共享权重:

$$W_U = E. \quad (14)$$

这样 logits 可写成:

$$z_{t,i} = E[i]^\top \mathbf{h}_t^L + b_i. \quad (15)$$

解释: 输出 token i 的 logit 是 hidden state 与 token embedding 的相似度。

6 Transformer Block 总览

6.1 Pre-LN Transformer block

现代 decoder-only LLM 多采用 Pre-LN 或 RMSNorm 变体:

$$\tilde{\mathbf{h}}^\ell = \mathbf{h}^\ell + \text{MHA}(\text{Norm}(\mathbf{h}^\ell)), \quad (16)$$

$$\mathbf{h}^{\ell+1} = \tilde{\mathbf{h}}^\ell + \text{FFN}(\text{Norm}(\tilde{\mathbf{h}}^\ell)). \quad (17)$$

Transformer 原始结构来源: [1]。Pre-LN 是后续稳定训练常见工程选择。
依赖关系:

Norm $\rightarrow$ Attention $\rightarrow$ Residual $\rightarrow$ Norm $\rightarrow$ FFN $\rightarrow$ Residual.

7 LayerNorm 与 RMSNorm

7.1 LayerNorm

对向量 $\mathbf{x} \in \mathbb{R}^d$:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad (18)$$

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2, \quad (19)$$

$$\text{LN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta. \quad (20)$$

来源: Layer Normalization [8]。

7.2 RMSNorm

RMSNorm 去掉均值中心化, 只除以 root mean square:

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}, \quad (21)$$

$$\text{RMSNorm}(\mathbf{x}) = \mathbf{g} \odot \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})}. \quad (22)$$

来源: RMSNorm [9]。

关系:

- LayerNorm 控制均值和方差。
- RMSNorm 只控制尺度, 计算更少, 很多 LLaMA-like 模型使用 RMSNorm。

8 Attention: 从 QK 到 Softmax

8.1 Q、K、V 的线性投影

对输入 hidden states $X \in \mathbb{R}^{T \times d_{\text{model}}}$:

$$Q = XW_Q, \quad W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad (23)$$

$$K = XW_K, \quad W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad (24)$$

$$V = XW_V, \quad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}. \quad (25)$$

含义:

- Query: 当前位置想找什么信息。
- Key: 每个上下文位置提供什么索引。
- Value: 真正被聚合的信息。

8.2 为什么是 $QK^\top$

第 i 个 query 与第 j 个 key 的相似度:

$$s_{ij} = \mathbf{q}_i^\top \mathbf{k}_j. \quad (26)$$

把所有位置一次性写成矩阵:

$$S = QK^\top, \quad S \in \mathbb{R}^{T \times T}. \quad (27)$$

缩放点积注意力:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (28)$$

来源: Transformer [1]。

为什么除以 $\sqrt{d_k}$:

若 q_i, k_i 独立且均值 0、方差 1, 则

$$\text{Var}(\mathbf{q}^\top \mathbf{k}) = \text{Var}\left(\sum_{r=1}^{d_k} q_r k_r\right) = d_k. \quad (29)$$

除以 $\sqrt{d_k}$ 后方差约为 1, 避免 softmax 输入随维度变大而过大, 导致梯度饱和。

8.3 Causal mask

Decoder-only LLM 不能看未来 token:

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (30)$$

于是:

$$\text{Attention}_{\text{causal}}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V. \quad (31)$$

8.4 Softmax 的数值稳定形式

直接计算 $\exp(z_i)$ 可能溢出, 通常减去最大值:

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}, \quad m = \max_j z_j. \quad (32)$$

这不改变结果, 因为分子分母同除 $\exp(m)$ 。

8.5 为什么 softmax/attention 内存受限

完整 attention score 的元素数:

$$\#S = B \cdot H \cdot T^2. \quad (33)$$

若用 fp16/bf16, 每个元素 2 bytes, 仅 score 矩阵显存读写量约为:

$$M_S \approx 2 \cdot BHT^2 \text{ bytes}. \quad (34)$$

实际还会有 softmax 概率矩阵、dropout mask、反向传播缓存等。因此长序列训练中, attention 往往受 HBM 带宽和中间矩阵 materialization 限制。

FlashAttention 的思想是不要显式 materialize 完整 $T \times T$ attention matrix, 而是分块做 online softmax。来源: FlashAttention [13]、FlashAttention-2 [14]。

8.6 FlashAttention 的 online softmax 递推

对一行 attention score 分块处理。维护:

$$m = \max_j s_j, \quad (35)$$

$$\ell = \sum_j \exp(s_j - m), \quad (36)$$

$$\mathbf{n} = \sum_j \exp(s_j - m) \mathbf{v}_j, \quad (37)$$

$$\mathbf{o} = \frac{\mathbf{n}}{\ell}. \quad (38)$$

当新块 B 到来:

$$m' = \max(m, \max_{j \in B} s_j), \quad (39)$$

$$\ell' = e^{m-m'} \ell + \sum_{j \in B} e^{s_j - m'}, \quad (40)$$

$$\mathbf{n}' = e^{m-m'} \mathbf{n} + \sum_{j \in B} e^{s_j - m'} \mathbf{v}_j, \quad (41)$$

$$\mathbf{o}' = \frac{\mathbf{n}'}{\ell'}. \quad (42)$$

这个递推保证结果等价于完整 softmax, 但显著减少 HBM 读写。来源: [13]。

9 Multi-Head、MQA 与 GQA

9.1 Multi-Head Attention

第 h 个 head:

$$\text{head}_h = \text{Attention}(XW_Q^h, XW_K^h, XW_V^h). \quad (43)$$

拼接所有 heads:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O. \quad (44)$$

来源: [1]。

9.2 Multi-Query Attention

MQA 让多个 query heads 共享同一组 K/V:

$$Q_h = XW_Q^h, \quad h = 1, \dots, H, \quad (45)$$

$$K = XW_K, \quad V = XW_V, \quad (46)$$

$$\text{head}_h = \text{Attention}(Q_h, K, V). \quad (47)$$

KV cache 从 H 组变成 1 组, 显著降低解码时显存带宽和 KV cache 大小。来源: Multi-Query Attention [15]。

9.3 Grouped-Query Attention

GQA 是 MHA 和 MQA 的折中。设 $G = H_{\text{kv}}$ 个 KV groups, query head h 使用 group:

$$g(h) = \left\lfloor \frac{hG}{H} \right\rfloor. \quad (48)$$

则:

$$\text{head}_h = \text{Attention}(Q_h, K_{g(h)}, V_{g(h)}). \quad (49)$$

来源: GQA [16]。

关系:

$$\text{MHA} : H_{\text{kv}} = H, \quad \text{GQA} : 1 < H_{\text{kv}} < H, \quad \text{MQA} : H_{\text{kv}} = 1.$$

10 位置编码

10.1 Sinusoidal positional encoding

Transformer 原始位置编码:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad (50)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right). \quad (51)$$

输入变为:

$$\mathbf{h}_t^0 = E[x_t] + PE(t). \quad (52)$$

来源: [1]。

10.2 RoPE

RoPE 对 query/key 做位置相关旋转。对二维子空间:

$$R_{\theta, m} = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}. \quad (53)$$

位置 m 的 query 和位置 n 的 key:

$$\tilde{\mathbf{q}}_m = R_m \mathbf{q}_m, \quad \tilde{\mathbf{k}}_n = R_n \mathbf{k}_n. \quad (54)$$

注意力分数:

$$\tilde{\mathbf{q}}_m^\top \tilde{\mathbf{k}}_n = \mathbf{q}_m^\top R_m^\top R_n \mathbf{k}_n = \mathbf{q}_m^\top R_{n-m} \mathbf{k}_n. \quad (55)$$

所以 RoPE 天然把相对位置 $n - m$ 注入 attention score。来源: RoFormer/RoPE [11]。

10.3 ALiBi

ALiBi 直接给 attention score 加线性距离偏置:

$$s_{h,i,j} = \frac{\mathbf{q}_{h,i}^\top \mathbf{k}_{h,j}}{\sqrt{d_h}} - m_h(i - j), \quad j \leq i, \quad (56)$$

其中 m_h 是 head-specific slope。来源: ALiBi [12]。

关系:

- Sinusoidal: 位置向量加到输入 hidden。
- RoPE: 位置旋转进 Q/K, 使 attention score 依赖相对距离。
- ALiBi: 直接对 attention score 加距离惩罚。

11 FFN、GELU 与 SwiGLU

11.1 标准 FFN

Transformer 每层还有 position-wise FFN:

$$\text{FFN}(\mathbf{x}) = W_2 \phi(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2. \quad (57)$$

原始 Transformer 使用 ReLU。来源: [1]。

11.2 GELU

GELU:

$$\text{GELU}(x) = x\Phi(x), \quad (58)$$

其中 Φ 是标准正态分布 CDF。常用近似:

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right] \right). \quad (59)$$

11.3 SwiGLU

现代 LLaMA-like 模型常用 SwiGLU:

$$\text{SiLU}(x) = x\sigma(x), \quad (60)$$

$$\text{SwiGLU}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}W_1) \odot \mathbf{x}W_3) W_2. \quad (61)$$

来源: GLU variants [10]。

解释: W_3 分支像 gate, 控制 W_1 分支的信息通过量。

12 Softmax 交叉熵的梯度

设 logits $\mathbf{z} \in \mathbb{R}^V$, 概率 $\mathbf{p} = \text{softmax}(\mathbf{z})$, 真实标签 one-hot 为 $\mathbf{y}$:

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^V y_i \log p_i. \quad (62)$$

先求:

$$\log p_k = z_k - \log \sum_j e^{z_j}. \quad (63)$$

对 z_i 求导:

$$\frac{\partial \log p_k}{\partial z_i} = \mathbf{1}[i = k] - p_i. \quad (64)$$

因此:

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_i} = - \sum_k y_k \frac{\partial \log p_k}{\partial z_i} \quad (65)$$

$$= - \sum_k y_k (\mathbf{1}[i = k] - p_i) \quad (66)$$

$$= p_i - y_i. \quad (67)$$

这是 LLM 训练中最重要的梯度公式之一:

$$\boxed{\nabla_z \mathcal{L}_{\text{CE}} = \mathbf{p} - \mathbf{y}}.$$

关系:

logits $\rightarrow$ softmax $\rightarrow$ cross entropy $\rightarrow \mathbf{p} - \mathbf{y} \rightarrow$ backprop into Transformer.

12.1 Label smoothing

真实 one-hot $\mathbf{y}$ 可替换为:

$$\mathbf{y}' = (1 - \epsilon)\mathbf{y} + \frac{\epsilon}{V}\mathbf{1}. \quad (68)$$

则梯度变为:

$$\nabla_z \mathcal{L} = \mathbf{p} - \mathbf{y}'. \quad (69)$$

13 优化器和训练动态

13.1 SGD

最基础更新:

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta_t). \quad (70)$$

13.2 Adam

Adam 维护一阶、二阶矩估计：

$$\mathbf{g}_t = \nabla_{\theta} \mathcal{L}(\theta_t), \quad (71)$$

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (72)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2. \quad (73)$$

偏差修正：

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}. \quad (74)$$

更新：

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \quad (75)$$

13.3 AdamW

AdamW 把 weight decay 从梯度里解耦：

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} - \eta_t \lambda \theta_t. \quad (76)$$

来源：AdamW [17]。

13.4 Gradient clipping

若全局梯度范数过大：

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min \left(1, \frac{c}{\|\mathbf{g}\|_2} \right). \quad (77)$$

作用：防止梯度爆炸。

13.5 Warmup + cosine decay

常见学习率：

$$\eta(t) = \begin{cases} \eta_{\max} \frac{t}{T_{\text{warmup}}}, & t < T_{\text{warmup}}, \\ \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi(t - T_{\text{warmup}})}{T_{\text{total}} - T_{\text{warmup}}} \right), & t \geq T_{\text{warmup}}. \end{cases} \quad (78)$$

14 Scaling Laws 与 Chinchilla

14.1 经验损失分解

Scaling laws 常把 loss 写成模型大小 N 、数据 token 数 D 的函数：

$$L(N, D) \approx L_\infty + \frac{A}{N^\alpha} + \frac{B}{D^\beta}. \quad (79)$$

来源：Kaplan et al. scaling laws [18], Chinchilla [19]。

14.2 训练计算量近似

Transformer dense LM 的训练 FLOPs 常用近似：

$$C \approx 6ND, \quad (80)$$

其中 N 是非 embedding 参数量， D 是训练 token 数。来源：Chinchilla [19]。
关系：

C 固定 $\implies$ 选择 N, D 的配比 $\implies$ compute-optimal training.

直观结论：

- 模型太大、数据太少：under-trained。
- 数据太多、模型太小：capacity 不足。
- Chinchilla 强调在固定 compute 下需要更多 tokens、更小但训练充分的模型。

15 SFT、Reward Model、RLHF 和 DPO

15.1 Supervised Fine-Tuning

给定 prompt x 和参考回答 $y_{1:T}$ ：

$$\mathcal{L}_{\text{SFT}} = - \sum_{t=1}^T \log \pi_\theta(y_t \mid x, y_{<t}). \quad (81)$$

它和预训练 NLL 形式一样，只是数据变成 instruction-response。

15.2 Reward model

RLHF 中，人类偏好数据常为 (x, y_w, y_l) : y_w 优于 y_l 。Bradley-Terry 形式：

$$P(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)). \quad (82)$$

reward model loss:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l)} \log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)). \quad (83)$$

来源：InstructGPT/RLHF [20]。

15.3 PPO for RLHF

PPO 的 clipped objective:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \quad (84)$$

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]. \quad (85)$$

来源：PPO [21]。

LLM RLHF 通常还加 KL penalty，避免策略偏离 reference model:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x)} [r_\phi(x, y) - \beta D_{\text{KL}}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))]. \quad (86)$$

来源：InstructGPT [20]。

15.4 DPO

DPO 直接从偏好对优化策略，不显式训练 reward model 做 RL rollouts:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \log \sigma \left(\beta [\log \pi_\theta(y_w \mid x) - \log \pi_{\text{ref}}(y_w \mid x) \right. \quad (87)$$

$$\left. - \log \pi_\theta(y_l \mid x) + \log \pi_{\text{ref}}(y_l \mid x)] \right). \quad (88)$$

来源：DPO [22]。

关系：

$$\text{Pretrain} \rightarrow \text{SFT} \rightarrow \begin{cases} \text{Reward Model} + \text{PPO} \\ \text{DPO} \end{cases} \rightarrow \text{Aligned LLM.}$$

16 LoRA 与参数高效微调

16.1 低秩适配

冻结原始权重 $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, 只学习低秩增量:

$$W' = W_0 + \Delta W, \quad \Delta W = \frac{\alpha}{r} B A, \quad (89)$$

其中:

$$B \in \mathbb{R}^{d_{\text{out}} \times r}, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad r \ll \min(d_{\text{in}}, d_{\text{out}}). \quad (90)$$

前向:

$$\mathbf{y} = W_0 \mathbf{x} + \frac{\alpha}{r} B A \mathbf{x}. \quad (91)$$

来源: LoRA [23]。

关系:

大矩阵全量更新 $\implies$ 低秩增量更新 $\implies$ 显著减少可训练参数和优化器状态.

17 MoE: Dense 与 Sparse

17.1 Dense FFN

Dense Transformer 每个 token 都经过同一个 FFN:

$$\mathbf{y}_t = \text{FFN}(\mathbf{x}_t). \quad (92)$$

每个 token 激活同一组参数。

17.2 Sparse MoE

有 E 个 experts, router 输出:

$$\mathbf{p}_t = \text{softmax}(W_r \mathbf{x}_t). \quad (93)$$

选择 top- k experts:

$$\mathcal{E}_t = \text{TopK}(\mathbf{p}_t, k). \quad (94)$$

输出:

$$\mathbf{y}_t = \sum_{e \in \mathcal{E}_t} p_{t,e} \text{Expert}_e(\mathbf{x}_t). \quad (95)$$

来源：GShard [24]、Switch Transformer [25]。

17.3 Switch Transformer top-1

Switch Transformer 使用 top-1 routing:

$$e_t^* = \arg \max_e p_{t,e}, \quad \mathbf{y}_t = p_{t,e_t^*} \text{Expert}_{e_t^*}(\mathbf{x}_t). \quad (96)$$

17.4 负载均衡损失

设 f_e 是分配到 expert e 的 token 比例, P_e 是 router 给 expert e 的平均概率, Switch 的辅助损失:

$$\mathcal{L}_{\text{aux}} = \alpha E \sum_{e=1}^E f_e P_e. \quad (97)$$

来源：Switch Transformer [25]。

关系:

- Dense: 所有 token 都走相同参数, 计算稳定但参数利用密集。
- Sparse MoE: 每个 token 只激活少数 experts, 总参数可很大, 单 token 计算可控。
- Expert Parallelism: 把 experts 分布到不同 GPU, 需要 all-to-all token dispatch/combine。

18 推理：从 logits 到 token

18.1 Temperature

训练时使用 $\text{softmax}(\mathbf{z})$, 推理时可调 temperature τ :

$$p_\tau(i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}. \quad (98)$$

- $\tau < 1$: 分布更尖锐, 更保守。
- $\tau > 1$: 分布更平, 更随机。

18.2 Greedy decoding

$$x_{t+1} = \arg \max_i p_\theta(i \mid x_{\leq t}). \quad (99)$$

18.3 Top-k sampling

只保留概率最高的 k 个 token:

$$S_k = \text{TopK}(\mathbf{p}, k), \quad \tilde{p}_i = \begin{cases} \frac{p_i}{\sum_{j \in S_k} p_j}, & i \in S_k, \\ 0, & i \notin S_k. \end{cases} \quad (100)$$

18.4 Top-p / nucleus sampling

选择最小集合 S_p , 使累计概率超过阈值 p :

$$S_p = \min \left\{ S : \sum_{i \in S} p_i \geq p \right\}. \quad (101)$$

归一化:

$$\tilde{p}_i = \begin{cases} \frac{p_i}{\sum_{j \in S_p} p_j}, & i \in S_p, \\ 0, & i \notin S_p. \end{cases} \quad (102)$$

18.5 Beam search

维护 K 条候选序列, 序列分数:

$$\text{score}(y_{1:T}) = \sum_{t=1}^T \log p_\theta(y_t \mid x, y_{<t}). \quad (103)$$

常加长度惩罚:

$$\text{score}_{\text{lp}}(y) = \frac{1}{T^\alpha} \sum_{t=1}^T \log p_\theta(y_t \mid x, y_{<t}). \quad (104)$$

19 KV Cache 与解码复杂度

19.1 没有 KV cache 的代价

生成第 t 个 token 时, 如果每一步都重算整个前缀, attention 总成本约为:

$$\sum_{t=1}^T O(t^2 d) = O(T^3 d). \quad (105)$$

实际 decoder 如果每步输入完整 prefix, 会重复计算历史 K/V。

19.2 有 KV cache 的增量解码

第 ℓ 层缓存历史:

$$K_{\leq t}^\ell = \text{Concat}(K_{\leq t-1}^\ell, K_t^\ell), \quad V_{\leq t}^\ell = \text{Concat}(V_{\leq t-1}^\ell, V_t^\ell). \quad (106)$$

新 token 只计算 query:

$$\mathbf{o}_t^\ell = \text{softmax} \left(\frac{\mathbf{q}_t^\ell (K_{\leq t}^\ell)^\top}{\sqrt{d_h}} \right) V_{\leq t}^\ell. \quad (107)$$

每步 attention 约 $O(td)$, 整段生成约:

$$\sum_{t=1}^T O(td) = O(T^2 d). \quad (108)$$

19.3 KV cache 显存公式

假设 dtype bytes 为 b , 层数 L , batch 中总上下文 token 数 T_{tot} :

$$M_{\text{KV}} = 2 \cdot L \cdot T_{\text{tot}} \cdot H_{\text{kv}} \cdot d_h \cdot b. \quad (109)$$

其中 2 表示 K 和 V。对比:

$$M_{\text{KV}}^{\text{MHA}} : M_{\text{KV}}^{\text{GQA}} : M_{\text{KV}}^{\text{MQA}} = H : G : 1. \quad (110)$$

19.4 PagedAttention

PagedAttention 把 KV cache 分成固定大小 blocks, 而不是为每个 request 分配连续大块内存。设 block size 为 B_{blk} , 序列 r 的 token 数为 T_r , 需要 block 数:

$$n_r = \left\lceil \frac{T_r}{B_{\text{blk}}} \right\rceil. \quad (111)$$

逻辑位置 t 映射到:

$$\text{block}(t) = \left\lfloor \frac{t}{B_{\text{blk}}} \right\rfloor, \quad \text{offset}(t) = t \bmod B_{\text{blk}}. \quad (112)$$

这样可以减少 KV cache 碎片, 并支持 prefix sharing/copy-on-write 等。来源: vLLM/PagedAttention [26]。

关系:

KV cache $\rightarrow$ 显存成为核心瓶颈 $\rightarrow$ PagedAttention/Block manager $\rightarrow$ continuous batching 和高吞吐服

20 Speculative Decoding

Speculative decoding 使用小 draft model q 先生成候选, 再由大 target model p 验证。
对 draft token x , 接受概率:

$$\alpha(x) = \min\left(1, \frac{p(x)}{q(x)}\right). \quad (113)$$

若拒绝, 则从修正分布采样:

$$p'(x) \propto \max(0, p(x) - q(x)). \quad (114)$$

来源: Speculative Decoding [27]。

直观关系:

draft model 快速提出 token $\rightarrow$ target model 并行验证多个 token $\rightarrow$ 减少大模型解码步数.

21 量化公式

21.1 Affine quantization

把浮点 x 量化为整数 q :

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right). \quad (115)$$

反量化:

$$\hat{x} = s(q - z). \quad (116)$$

其中 s 是 scale, z 是 zero point.

21.2 Symmetric quantization

对称量化常设 $z = 0$:

$$s = \frac{\max |x|}{2^{b-1} - 1}, \quad q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right), -2^{b-1}, 2^{b-1} - 1\right). \quad (117)$$

来源可参考 LLM.int8 [28]、GPTQ [29]。

关系:

低 bit $\rightarrow$ 显存/带宽下降 $\rightarrow$ 反量化或量化 kernel $\rightarrow$ 吞吐提升但精度可能下降.

22 分布式训练/推理的核心公式

22.1 Data Parallel

R 个 worker 各算一份梯度:

$$\mathbf{g} = \frac{1}{R} \sum_{r=1}^R \mathbf{g}^{(r)}. \quad (118)$$

然后每个 worker 使用同一个平均梯度更新参数。

22.2 Tensor Parallel: 列切分

线性层 $Y = XW$, 按列切:

$$W = [W_1, W_2, \dots, W_R]. \quad (119)$$

每张卡计算:

$$Y_r = XW_r. \quad (120)$$

最后 all-gather:

$$Y = [Y_1, Y_2, \dots, Y_R]. \quad (121)$$

22.3 Tensor Parallel: 行切分

按行切:

$$X = [X_1, \dots, X_R], \quad W = \begin{bmatrix} W_1 \\ \vdots \\ W_R \end{bmatrix}. \quad (122)$$

每张卡计算局部结果:

$$Y_r = X_r W_r. \quad (123)$$

最后 all-reduce:

$$Y = \sum_{r=1}^R Y_r. \quad (124)$$

22.4 Expert Parallel

MoE 中 expert 分布在不同 GPU。第 t 个 token 被 router 分配到 expert e_t :

$$e_t = \arg \max_e p_{t,e}. \quad (125)$$

系统需要执行:

token dispatch $\rightarrow$ expert local compute $\rightarrow$ token combine.

数学上是式 (95), 系统上是 all-to-all communication。

23 Multimodal 与 Diffusion 相关公式

这一节不是纯 LLM 必需, 但对理解 vLLM-Omni、Any-to-Any multimodal models 很重要。

23.1 Vision patch embedding

图像 $I \in \mathbb{R}^{H \times W \times C}$ 切成 N 个 patch, 每个 patch 展平为 $\mathbf{p}_i$:

$$\mathbf{v}_i = W_p \mathbf{p}_i + \mathbf{b}_p. \quad (126)$$

然后把 image tokens 和 text tokens 拼接:

$$X = [X_{\text{text}}; X_{\text{image}}]. \quad (127)$$

23.2 CLIP 对比学习损失

图文 embedding:

$$\mathbf{u}_i = f_{\text{text}}(t_i), \quad \mathbf{v}_i = f_{\text{image}}(I_i). \quad (128)$$

相似度:

$$s_{ij} = \frac{\mathbf{u}_i^\top \mathbf{v}_j}{\tau}. \quad (129)$$

image-to-text loss:

$$\mathcal{L}_{i2t} = -\frac{1}{B} \sum_i \log \frac{\exp(s_{ii})}{\sum_j \exp(s_{ij})}. \quad (130)$$

text-to-image 同理, 最终:

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}(\mathcal{L}_{i2t} + \mathcal{L}_{t2i}). \quad (131)$$

来源: CLIP [30]。

23.3 DDPM forward process

扩散模型前向加噪:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t I). \quad (132)$$

令 $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, 可直接采样:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t)I). \quad (133)$$

即:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I). \quad (134)$$

来源: DDPM [31]。

23.4 噪声预测训练目标

模型预测噪声:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} [\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c)\|_2^2]. \quad (135)$$

DiT 把 Transformer 用作 diffusion backbone。来源: DiT [32]。

23.5 Classifier-Free Guidance

有条件预测 $\epsilon_\theta(\mathbf{x}_t, t, c)$ 和无条件预测 $\epsilon_\theta(\mathbf{x}_t, t, \emptyset)$:

$$\hat{\epsilon} = \epsilon_\theta(\mathbf{x}_t, t, \emptyset) + w [\epsilon_\theta(\mathbf{x}_t, t, c) - \epsilon_\theta(\mathbf{x}_t, t, \emptyset)]. \quad (136)$$

来源: Classifier-Free Guidance [33]。

和 vLLM-Omni 的关系:

- AR stage 可产生 text/hidden/KV 条件。
- DiT stage 消费这些条件生成 image/video/audio。
- CFG companion requests 和 KV transfer 是系统实现层面的对应物。

24 总关联图

24.1 训练侧

$$\begin{aligned}
 \text{raw text} &\xrightarrow{\text{tokenizer}} x_{1:T} \xrightarrow{E} X \\
 &\xrightarrow{\text{Transformer blocks}} \mathbf{h}_{1:T}^L \xrightarrow{W_U} \mathbf{z}_{1:T} \xrightarrow{\text{softmax}} p_{\theta}(x_{t+1} \mid x_{\leq t}) \\
 &\xrightarrow{\text{CE/NLL}} \mathcal{L} \xrightarrow{\text{backprop}} \nabla_{\theta} \mathcal{L} \xrightarrow{\text{AdamW}} \theta_{t+1}.
 \end{aligned}$$

24.2 推理侧

$$\begin{aligned}
 \text{prompt} &\xrightarrow{\text{prefill}} K_{\leq T}, V_{\leq T}, \mathbf{h}_T \\
 &\xrightarrow{\text{decode step}} \mathbf{z}_T \xrightarrow{\text{temperature/top-k/top-p}} x_{T+1} \\
 &\xrightarrow{\text{append KV cache}} K_{\leq T+1}, V_{\leq T+1} \xrightarrow{\text{repeat}} \text{generated text}.
 \end{aligned}$$

24.3 系统侧

请求队列 $\rightarrow$ batching/scheduling $\rightarrow$ KV cache block allocation
 $\rightarrow$ GPU kernels: attention/FFN/sampling $\rightarrow$ streaming output.

vLLM/SGLang 主要优化的是这一层，而不是改变语言建模目标本身。

25 公式学习建议

建议按下面顺序掌握：

1. 先吃透式 (1)、(3)、(13)、(67)。
2. 再看 Transformer block：式 (17)、(28)、(31)。
3. 再看现代模型变化：RoPE 式 (55)、RMSNorm 式 (22)、SwiGLU 式 (61)。
4. 然后看训练工程：AdamW 式 (76)、Scaling Laws 式 (79)、计算量式 (80)。
5. 微调对齐：SFT 式 (81)、Reward Model 式 (83)、PPO 式 (85)、DPO 式 (88)。
6. 推理系统：KV cache 式 (107)、显存式 (109)、PagedAttention block mapping。
7. vLLM-Omni：再补 MoE 式 (95)、diffusion loss 式 (135)、CFG 式 (136)。

参考文献

- [1] Ashish Vaswani et al. Attention Is All You Need. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
- [2] Jacob Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. 2018. <https://arxiv.org/abs/1810.04805>
- [3] Alec Radford et al. Language Models are Unsupervised Multitask Learners. OpenAI, 2019. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- [4] Tom Brown et al. Language Models are Few-Shot Learners. NeurIPS 2020. <https://arxiv.org/abs/2005.14165>
- [5] Rico Sennrich, Barry Haddow, Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. 2015. <https://arxiv.org/abs/1508.07909>
- [6] Taku Kudo, John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. 2018. <https://aclanthology.org/D18-2012/>
- [7] Taku Kudo. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. 2018. <https://arxiv.org/abs/1804.10959>
- [8] Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E. Hinton. Layer Normalization. 2016. <https://arxiv.org/abs/1607.06450>
- [9] Biao Zhang, Rico Sennrich. Root Mean Square Layer Normalization. 2019. <https://arxiv.org/abs/1910.07467>
- [10] Noam Shazeer. GLU Variants Improve Transformer. 2020. <https://arxiv.org/abs/2002.05202>
- [11] Jianlin Su et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021. <https://arxiv.org/abs/2104.09864>
- [12] Ofir Press, Noah A. Smith, Mike Lewis. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. 2021. <https://arxiv.org/abs/2108.12409>
- [13] Tri Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022. <https://arxiv.org/abs/2205.14135>

- [14] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. 2023. <https://arxiv.org/abs/2307.08691>
- [15] Noam Shazeer. Fast Transformer Decoding: One Write-Head is All You Need. 2019. <https://arxiv.org/abs/1911.02150>
- [16] Joshua Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. 2023. <https://arxiv.org/abs/2305.13245>
- [17] Ilya Loshchilov, Frank Hutter. Decoupled Weight Decay Regularization. 2017. <https://arxiv.org/abs/1711.05101>
- [18] Jared Kaplan et al. Scaling Laws for Neural Language Models. 2020. <https://arxiv.org/abs/2001.08361>
- [19] Jordan Hoffmann et al. Training Compute-Optimal Large Language Models. 2022. <https://arxiv.org/abs/2203.15556>
- [20] Long Ouyang et al. Training language models to follow instructions with human feedback. 2022. <https://arxiv.org/abs/2203.02155>
- [21] John Schulman et al. Proximal Policy Optimization Algorithms. 2017. <https://arxiv.org/abs/1707.06347>
- [22] Rafael Rafailov et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. 2023. <https://arxiv.org/abs/2305.18290>
- [23] Edward J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. 2021. <https://arxiv.org/abs/2106.09685>
- [24] Dmitry Lepikhin et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. 2020. <https://arxiv.org/abs/2006.16668>
- [25] William Fedus, Barret Zoph, Noam Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. 2021. <https://arxiv.org/abs/2101.03961>
- [26] Woosuk Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. 2023. <https://arxiv.org/abs/2309.06180>
- [27] Yaniv Leviathan, Matan Kalman, Yossi Matias. Fast Inference from Transformers via Speculative Decoding. 2023. <https://arxiv.org/abs/2211.17192>
- [28] Tim Dettmers et al. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. 2022. <https://arxiv.org/abs/2208.07339>

- [29] Elias Frantar et al. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. 2022. <https://arxiv.org/abs/2210.17323>
- [30] Alec Radford et al. Learning Transferable Visual Models From Natural Language Supervision. 2021. <https://arxiv.org/abs/2103.00020>
- [31] Jonathan Ho, Ajay Jain, Pieter Abbeel. Denoising Diffusion Probabilistic Models. 2020. <https://arxiv.org/abs/2006.11239>
- [32] William Peebles, Saining Xie. Scalable Diffusion Models with Transformers. 2022. <https://arxiv.org/abs/2212.09748>
- [33] Jonathan Ho, Tim Salimans. Classifier-Free Diffusion Guidance. 2022. <https://arxiv.org/abs/2207.12598>